

SimpleRNN Class Implementation

rnn.cpp

Gary Hobson

Creating the SimpleRNN Class

The full implementation of the SimpleRNN class is shown below with IDE-style syntax highlighting (blue keywords, green comments, red strings, teal types).

```
1 // This line tells the compiler: "Include the declarations from rnn.h"
2 // Without this, the compiler would not know what SimpleRNN is, what its
   member variables are,
3 // or what functions it has declared.
4 #include "rnn.h"
5
6
7 //
8
9 // CONSTRUCTOR
10 // This is the function that gets called when you write: SimpleRNN
   my_rnn(1, 32);
11 // It creates a new RNN object and sets up its internal memory (weights
   & biases)
12 //
13
14 SimpleRNN::SimpleRNN(int in_size, int hid_size) // Parameters: input
   size & desired hidden layer size
15 : input_size(in_size), // Store how many
   numbers come in each timestep (usually 1 = univariate)
16 hidden_size(hid_size), // Store how many
   "memory cells" / neurons this RNN has in its hidden layer
   (controls capacity)
17
18 // Create the three main weight matrices with correct dimensions
19 // Wxh = "input to hidden" weights turns input vector into
   hidden vector contribution
20 Wxh(hid_size, in_size), // Rows = hidden
   neurons, Columns = input features
21
22 // Whh = "hidden to hidden" = recurrent weights lets the RNN
   remember previous state
23 Whh(hid_size, hidden_size), // Square matrix:
   hidden hidden
```

```

22
23 // Why = "hidden to output" weights      turns final hidden state
    into prediction
24 // We use 1 output because we're predicting one number (next power
    value)
25 Why(1, hid_size),                          // 1 row (one
    output), hidden_size columns
26
27 // Bias vectors      small numbers added after matrix
    multiplication (helps model fit better)
28 bh(hid_size),                          // One bias per
    hidden neuron
29 by(1)                                  // One bias for the
    single output
30 {
31 // The body is empty because we used the initializer list above
    (more efficient)
32 // No extra work needed after setting sizes
33 }
34
35
36 //
37 // Initializes all weights and biases with small random numbers
38 // Very important: good initialization prevents the network from
    starting with crazy values
39 //
40 void SimpleRNN::initialize_weights(double stddev) // stddev = how
    spread out the random numbers should be (usually small like 0.1)
41 {
42 // Create a source of true randomness (uses hardware if available)
    std::random_device rd;
43
44 // Create a high-quality random number engine (Mersenne Twister)
    std::mt19937 gen(rd());
45
46 // Create a normal (Gaussian) distribution: mean=0, standard
    deviation = stddev
47 // Most values will be between -3*stddev and +3*stddev
    std::normal_distribution<double> dist(0.0, stddev);
48
49 //
50
51 // Lambda function that fills ANY Eigen matrix with random numbers
52 // This is like a mini-function we can reuse
53 //
54
55 std::function<void(MatrixXd&)> fill_matrix = [&](MatrixXd& mat)
56 {
57

```

```

58     // Loop through every row
59     for (int i = 0; i < mat.rows(); ++i)
60     {
61         // Loop through every column in that row
62         for (int j = 0; j < mat.cols(); ++j)
63         {
64             // Assign a random number from our distribution
65             mat(i, j) = dist(gen);
66         }
67     }
68 };
69
70 // Similar lambda, but for vectors (1D arrays)
71 std::function<void(VectorXd&)> fill_vector = [&](VectorXd& vec)
72 {
73     for (int i = 0; i < vec.size(); ++i)
74     {
75         vec(i) = dist(gen);
76     }
77 };
78
79 // Now actually fill every weight matrix and bias vector
80 fill_matrix(Wxh); // Input hidden weights
81 fill_matrix(Whh); // Recurrent (memory) weights
82 fill_matrix(Why); // Hidden prediction weights
83 fill_vector(bh); // Hidden layer bias
84 fill_vector(by); // Output bias
85 }
86
87
88 //
89 // FORWARD STEP the heart of the RNN computes one time step
90 // Takes current input x_t and previous memory h_prev
91 // Returns the new memory state h_t
92 //
93 VectorXd SimpleRNN::forward_step(const VectorXd& x_t, const VectorXd&
94 h_prev) const
95 {
96     // Step 1: Compute linear combination
97     // Wxh * x_t how much the current input contributes to
98     // hidden state
99     // Whh * h_prev how much the previous memory contributes (this
100     // is the "recurrent" part)
101     // + bh add bias to shift the result
102     // Result so far: a vector of size hidden_size with raw scores
103
104     // Step 2: Apply tanh activation function to every element
105     // tanh squashes values to range [-1, 1]

```

```

103 // This prevents numbers from growing too large or too small over
    // many timesteps
104 // .unaryExpr applies the lambda to every single element of the
    // vector
105 VectorXd hidden = (Wxh * x_t + Whh * h_prev + bh)
106                 .unaryExpr([](double x)
107                 {
108                     return std::tanh(x); // std::tanh is the
                                           // hyperbolic tangent function
109                 });
110
111 // Return the new hidden state (memory) for this timestep
112 return hidden;
113 }
114
115 //
116 //
117 // FORWARD SEQUENCE runs the RNN over many timesteps (whole sequence)
118 // Fills two output vectors by reference so caller can see all hidden
    // states & predictions
119 //
120 void SimpleRNN::forward_sequence(const std::vector<VectorXd>& inputs,
121                                 std::vector<VectorXd>& hidden_states,
122                                 std::vector<VectorXd>& outputs) const
123 {
124     // Clear any old data that might be in these vectors
125     hidden_states.clear();
126     outputs.clear();
127
128     // Start with zero memory (no previous information before first
    // input)
129     VectorXd h = VectorXd::Zero(hidden_size); // all elements = 0.0
130
131     // Loop through every input timestep
132     for (const VectorXd& x : inputs)
133     {
134         // Compute next memory state using current input and previous
            // memory
135         h = forward_step(x, h);
136
137         // Save this hidden state (useful later for training or
            // visualization)
138         hidden_states.push_back(h);
139
140         // Compute prediction for this timestep
141         // Why * h project hidden state to output space
142         // + by add output bias
143         VectorXd y_pred = Why * h + by;

```

```

144         // Save the prediction
145         outputs.push_back(y_pred);
146     }
147     // After the loop:
148     // hidden_states has one entry per input timestep
149     // outputs has one prediction per input timestep
150 }
151
152
153
154 //
155 // LOSS FUNCTION      measures how wrong our predictions are
156 // Uses Mean Squared Error (MSE)      very common for regression problems
157 //
158 double SimpleRNN::compute_loss(const std::vector<VectorXd>& targets,
159                                const std::vector<VectorXd>& predictions)
160                                const
161 {
162     // Safety check: we must have exactly the same number of targets and
163     // predictions
164     if (targets.size() != predictions.size())
165     {
166         std::cout << "Target/prediction size mismatch\n";    // Print
167         warning
168         return 0.0;                                           // Return
169         invalid value
170     }
171
172     double total_loss = 0.0;    // We'll add up all squared errors here
173
174     // Loop over every timestep we have both target and prediction for
175     for (size_t i = 0; i < targets.size(); ++i)
176     {
177         // Compute difference: target - prediction (vector subtraction)
178         VectorXd diff = targets[i] - predictions[i];
179
180         // squaredNorm() = sum of squares of all elements
181         // Since our output is size 1, this is basically (target -
182         // pred)
183         total_loss += diff.squaredNorm();
184     }
185
186     // Return average squared error (Mean Squared Error)
187     // This number is what we want to make SMALLER during training
188     return total_loss / targets.size();
189 }

```